

Mapping Lake Temperature Model

Purpose

- Find and identify patterns in how changing the constants of `elmtest.exe`, an environmental model, affects its outputs.

What is elmtest.exe?

- Fortran binary
- Compiled in a docker image from [Docker4SPEL](#)
- Environmental model

elmtest.exe interface

- Inputs: lakeparams.txt
- Outputs: cpu_LakeTemperatureTest.txt

lakeparams.txt

CONTAINER:/app/elmtest/lakeparams.txt

```
betavis  
1.0  
za_lake  
2.0  
n2min  
7.5e-05  
tdmax  
275.0  
pudz  
11.0  
mixfact  
0.5  
depthcrit  
25.0
```

cpu_LakeTemperatureTest.txt

CONTAINER:/app/elmtest/cpu_LakeTemperatureTest.txt (Default Parameters)

col_ws%h2osno

0.0000000000000000

0.0000000000000000

0.0000000000000000

0.0000000000000000

0.0000000000000000

0.0000000000000000

0.0000000000000000

0.0000000000000000

0.0000000000000000

0.0000000000000000

0.0000000000000000

0.0000000000000000

0.0000000000000000

0.0000000000000000

0.0000000000000000

0.0000000000000000

0.0000000000000000

0.0000000000000000

- - - ... - - -

col_es%hc_soi

1.0000000000000000E+036

1.0000000000000000E+036

1.0000000000000000E+036

1.0000000000000000E+036

1.0000000000000000E+036

1.0000000000000000E+036

1.0000000000000000E+036

1.0000000000000000E+036

1.0000000000000000E+036

1.0000000000000000E+036

1.0000000000000000E+036

1.0000000000000000E+036

1.0000000000000000E+036

1.0000000000000000E+036

1.0000000000000000E+036

1.0000000000000000E+036

15270.67984628235

1.0000000000000000E+036

- - - ... - - -

Approach

- Write change to lakeparams.txt
- Run elmtest.exe
- Compare cpu_LakeTemperatureTest.txt to cpu_LakeTemperatureRef.txt
- Repeat

Interface of mapping script

- Orders
 - Define a group of samples
- Output
 - Console output and/or csv output

Example Order: betavis_min_to_max.json

lake_temp_work/lake_temperature_mapper/mapping_orders/betavis_min_to_max.json

```
{
  "samples":11,
  "ranges":{
    "betavis": ["r(0)", "r(1)"]
  }
}
```

Results in: [0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]

What does $r(t)$ mean?

Represents a value returned from performing an unbounded linear interpolation between the minimum and maximum size specified in FUT_lake_range.txt, with t as the time value.

Example Line Order: all_min_to_max.json

lake_temp_work/lake_temperature_mapper/mapping_orders/all_min_to_max.json

```
{
  "samples":11,
  "ranges": {
    "betavis":["r(0)", "r(1)"],
    "za_lake":["r(0)", "r(1)"],
    "n2min":["r(0)", "r(1)"],
    "tdmax":["r(0)", "r(1)"],
    "pudz":["r(0)", "r(1)"],
    "mixfact":["r(0)", "r(1)"],
    "depthcrit":["r(0)", "r(1)"]
  }
}
```

Interpolates between each range *simultaneously*, forming a line.

Example Box Order: betavis_za_lake_box.json

lake_temp_work/lake_temperature_mapper/mapping_orders/betavis_za_lake_box.json

```
{
  "ranges": {
    "betavis": ["r(0)", "r(1)", 5],
    "za_lake": ["r(0)", "r(1)", 5]
  }
}
```

Interpolates between each axis, then uses all combinations of those samples,
forming a box.

Example Console Output

CONTAINER:/app/scripts/cli.py running betavis_min_to_max order.

Executing order: betavis_min_to_max

11 samples using ranges:

Betavis: 0.0 -> 1.0

Set betavis to 0.0

lakestate_vars%betaprime_col

1.0	-> 0.0	by -1.0	at 16
1.0	-> 0.0	by -1.0	at 33
1.0	-> 0.0	by -1.0	at 50

- - - ... - - -

Set betavis to 0.1

lakestate_vars%betaprime_col

1.0	-> 0.1	by -0.9	at 16
1.0	-> 0.1	by -0.9	at 33
1.0	-> 0.1	by -0.9	at 50

- - - ... - - -

Set betavis to 1.0

No differences

Example CSV Output

CONTAINER:/app/scripts/cli.py running betavis_min_to_max order with -s flag.
CONTAINER:/app/mapping_output/betavis_min_to_max.csv

```
betavis,0.0  
lakestate_vars%betaprime_col  
1.0,0.0,-1.0,16  
1.0,0.0,-1.0,33  
1.0,0.0,-1.0,50
```

- - - ... - - -

```
betavis,0.1  
lakestate_vars%betaprime_col  
1.0,0.1,-0.9,16  
1.0,0.1,-0.9,33  
1.0,0.1,-0.9,50
```

- - - ... - - -

```
betavis,1.0
```

Development using fake_binary.exe

- The Docker container is stateless
- elmtest.exe depends on the environment inside the docker image
- fake_binary.exe was developed to imitate elmtest.exe but without depending on the docker image
 - Uses known functions to compute output, making testing easier

fake_LakeTemperatureTest.txt

lake_temp_work/lake_temperature_mapper/fake_binary/fake_LakeTemperatureTest.txt
(Default Parameters)

```
fakevar%sum
  314.500075 629.000150
  31.450007 157.250037
fakevar%product
  31.450007 157.250037
  5.671875 32.170166
```

Example Console Output with fake_binary

lake_temp_work/lake_temperature_mapper/scripts/cli.py running
betavis_min_to_max order.

Executing order: betavis_min_to_max
betavis: 0.0 -> 1.0, 11 samples

Set betavis to 0.0

fakevar%sum

314.500075	-> 313.500075	by -1.0	at 0
629.00015	-> 627.00015	by -2.0	at 1
31.450007	-> 31.350008	by -0.099999000000000004	at 2

- - - ... - - -

Set betavis to 0.1

fakevar%sum

314.500075	-> 313.600075	by -0.89999999999999773	at 0
629.00015	-> 627.20015	by -1.7999999999999545	at 1
31.450007	-> 31.360008	by -0.089998999999999883	at 2

- - - ... - - -

Set betavis to 1.0

No differences

Configuration of cli.py

- Configuration files necessary to account for differences between naming of elmtest.exe and fake_binary.exe files.
- Can generate a configuration file using configure_mapper.py

Configuration files

lake_temp_work/lake_temperature_mapper/config/mapper.conf

```
binary_path: fake_binary/fake_binary.exe
range_path: fake_binary/FUT_lake_range.txt
params_path: fake_binary/lakeparams.txt
defaults_path: fake_binary/lakeparams_defaults.txt
ref_output: fake_binary/fake_LakeTemperatureRef.txt
test_output: fake_binary/fake_LakeTemperatureTest.txt
order_directory: mapping_orders
output_directory: mapping_output
binary_args:
```

CONTAINER:/app/config/mapper.conf

```
binary_path: elmtest/elmtest.exe
range_path: elmtest/FUT_lake_range.txt
params_path: elmtest/lakeparams.txt
defaults_path: elmtest/lakeparams_defaults.txt
ref_output: elmtest/cpu_LakeTemperatureRef.txt
test_output: elmtest/cpu_LakeTemperatureTest.txt
order_directory: mapping_orders
output_directory: mapping_output
binary_args: 1
```